

Caching In Scala Using Cats Ref



By Katrina Petrevic

May 2026

What is Ref?

A concurrent mutable reference

Safe, pure, and composable

Ref — A concurrent mutable reference

Signature: `Ref[F[_], T]`

Safe shared state across fibers

Atomic updates without locks

Works inside IO — Think: AtomicReference

When to Choose In-Memory Caching

Fast Access

No network, no serialization. Microsecond latency in JVM memory

Hot Data

Frequently accessed data is cheap to keep in-memory

Single/Independent Instances

Cache duplication across servers is acceptable

When to Choose Cloud Caching

Shared State

Multiple servers need consistent cache

Persistence

Cache survives restarts with TTL

Larger Capacity

Store datasets larger than heap

Global Invalidation

Update once, all see instantly

Trade-Offs when Implementing Caching

Cold Start

Low Hit Rate

Invalidation

Memory Overhead

1. Cold Start

✗ Problem: Cache is empty on startup causing performance degradation

Why it happens: No historical data to serve on initialization

✓ Fixability: Inherent but mitigatable

- Warm-up data on startup (pre-load top data)
- Accept initial high latency

2. Low Cache Hit Rate

✗ Problem: Many unique data instances reduce cache reuse

Why it happens: Cache keys too granular

✓ Fixability: Yes, highly fixable!

- Use composite keys (user + category, not user + timestamp)
- Analyze access patterns to find reusable data

3. Cache Invalidation

 **Challenge:** Choosing between stale data (TTL) vs event driven

Why it's tricky: No one-size-fits-all solution; depends on freshness requirements

 **Fixability: Yes, with right strategy**

- Time-based (TTL): Cache expires after 5 min
- Event-driven: Invalidate on data state changes

4. Memory/Cost Overhead

✗ Problem: RAM is expensive and limited; Redis/CDN add operational costs

Why it happens: Unbounded cache growth + expensive infrastructure

✓ Fixability: Yes, controllable

- Set size limits + Least Recently Used (LRU) eviction
- Monitor utilization & alert on thresholds

Scenario - API Request & Response

Use Case

Caching on the Service layer

Goal

User wants immediate response on the UI page

```

object CacheService {
  final case class Request(id: String)
  final case class Response(value: String)
  private val ttl = Duration.ofMinutes(5)
  val cacheRef = Ref.unsafe(Map.empty)
  def cacheResponse(request: Request): IO[Response] = {
    def updateCache(cacheKey: String): IO[Response] = {
      for {
        cacheTime <- IO(Instant.now())
        response <- computeResponse(request)
        _ <- cacheRef.update(_>.updated(cacheKey, (response, cacheTime)))
      } yield response
    }
    for {
      currentTime <- IO(Instant.now())
      cacheKey = request.id
      cachedEntry <- cacheRef.get.map(mapOfRef => mapOfRef.get(cacheKey))
      response <- cachedEntry match {
        case Some((resp, cacheTime)) if isValid(cacheTime, currentTime) =>
          IO.pure(resp)
        case _ => updateCache(cacheKey)
      }
    } yield response
  }
  private def computeResponse(req: Request): IO[Response] =
    IO(Response(s"computed-${req.id}-${System.nanoTime()}"))
  private def isValid(cacheTime: Instant, now: Instant): Boolean =

```

Testing CacheService — Expected

```
test("CacheService returns cached response within TTL") {  
  val service = CacheService  
  val request = Request("someId")  
  for {  
    // First call: should compute and cache  
    response1 <- service.cacheResponse(request)  
    // Second call within TTL: should return cached  
    response2 <- service.cacheResponse(request)  
  } yield assert(response1 == response2)  
}
```

Why CacheService is Tricky to Test?

Challenge 1: Time-Dependent Behavior

Problem

Can't use real `Instant.now()` — tests become slow & flaky

Solution

Mock/inject time or use virtual time testing

Testing CacheService — PITFALL 1: Time

```
test("CacheService respects TTL - FLAKY") {  
  val service = CacheService  
  val request = Request("someId")  
  for {  
    response1 <- service.cacheResponse(request)  
    _ <- IO.sleep("Some really long time for TTL")  
    response2 <- service.cacheResponse(request)  
  } yield assert(response1 != response2)  
}
```


Challenge 2: Shared Mutable State

Problem

``Ref.unsafe(Map.empty)`` at global level causes state bleed

Issue

Tests interfere if cache isn't reset between runs

Testing CacheService — PITFALL 2: State Bleed

```
test("CacheService caches responses") {  
  val service = CacheService // Ref shared across test runs!  
  test("populates cache") {  
    for {  
      cache <- service.cacheResponse(service.Request("key1"))  
    } yield {assert(cache.value.nonEmpty)}  
  }  
  test("start with non-empty cache (state bleed)") {  
    for {  
      cache <- service.cacheRef.get  
    } yield {assert(cache.nonEmpty, "Cache should be empty at test start!")}  
  }  
}
```

Challenge 3: Concurrent Edge Cases

Race Condition

Time check + cache read/update not atomic

Consequence

Multiple fibers may compute same result independently

Testing CacheService — PITFALL 3: Concurrency

```
test("CacheService has race between get and update") {  
  val service = CacheService  
  val request = Request("shared-key")  
  for {  
    //start - Start execution of the source suspended in the IO context  
    response1 <- service.cacheResponse(request).start  
    response2 <- service.cacheResponse(request).start  
    //joinWithNever - Awaits the completion of the bound fiber and returns its res  
    r1 <- response1.joinWithNever  
    r2 <- response2.joinWithNever  
    cache <- CacheService.cacheRef.get  
  } yield {  
    // both fibers computed independently for same key  
    assert(r1 != r2)  
  }  
}
```

```

final case class CacheEntry[A](value: IO[A], cachedAt: Instant)
final case class Request(id: String)
final case class Response(value: String)
class CacheService(
  cacheRef: Ref[IO, Map[String, CacheEntry[Response]]],
  clock: Clock[IO]) {
  def cacheResponse(request: Request, ttl: FiniteDuration): IO[Response] = {
    for {
      timeNow <- clock.realTimeInstant
      cacheKey = hashRequest(request)
      ioResponse <- cacheRef.modify { cache =>
        cache.get(cacheKey) match {
          case Some(entry) if isValid(entry, timeNow, ttl) =>
            (cache, entry.value)
          case _ =>
            val newResponse = computeResponse(request)
            val updated = cache.updated(
              cacheKey,
              CacheEntry(newResponse, timeNow)
            )
            (updated, newResponse)
        }
      }
      response <- ioResponse
    } yield response
  }
  private def isValid(entry: CacheEntry[Response], now: Instant, ttl: FiniteDuration): Boolean =

```

Key Improvements

- **Clock injected** for testable, mockable time
- **Ref injected** for explicit dependency control and state lifecycle management
- **Atomic** `cacheRef.modify` Signature: $(f: A \Rightarrow (A, B)): F[B]$
 - update $\rightarrow$ atomic update, no returned value
 - modify $\rightarrow$ atomic update + atomic read/result
- `Ref.of` safer when injecting dependency

Summary - Key Takeaways

Prefer effect-wrapped caches (e.g., `Ref[IO, _]`) to maintain purity

Use atomic operations like `Ref.modify` for safe updates

Combine L1 + L2 caches for optimal latency and scalability

Test concurrency explicitly with virtual time and race condition scenarios

Questions?



Resources & Further Reading

Typelevel Cats Effect

Official documentation for Ref:

- Scala 2.x: <https://typelevel.org/cats-effect/docs/2.x/concurrency/ref>
- Scala 3.x: <https://typelevel.org/cats-effect/docs/std/ref>